# PRÁCTICA DE DESARROLLO

# CI/CD con GitHub Actions y Notion

*1. Introducción y Objetivos ..... 3*

*2. Herramientas y Prerrequisitos ..... 4*

*FASE 1 – Página Web de Login (HTML/CSS/JS) ..... 5*

*FASE 2 – Repositorio GitHub y primer commit ..... 6*

*FASE 3 – Validaciones y Testing con GitHub Actions ..... 8*

*TAREA FINAL – Copia automática de archivos a Amazon S3 ..... 18*

# 1. Introducción y Objetivos

Esta práctica guía a través de un ciclo DevOps completo y simplificado. Partiendo de una página de login básica, se construye un pipeline de integración continua (CI) y despliegue continuo (CD) .

**Objetivos generales:**

- Crear y versionar código con Git y GitHub.
- Automatizar pruebas y validaciones con GitHub Actions.
- Documentar el proyecto y gestionar tareas con Notion.

# 2. Herramientas y Prerrequisitos

## 2.1 Cuentas y servicios en la nube

- GitHub – cuenta gratuita en https://github.com
- Notion – cuenta gratuita en https://notion.so

## 2.2 Configuración inicial de Git

Si aún no se ha configurado ejecutar en terminal antes de empezar.

```
git config --global user.name  "Tu Nombre"
git config --global user.email "tu@email.com"
git config --global core.editor "code --wait"
```

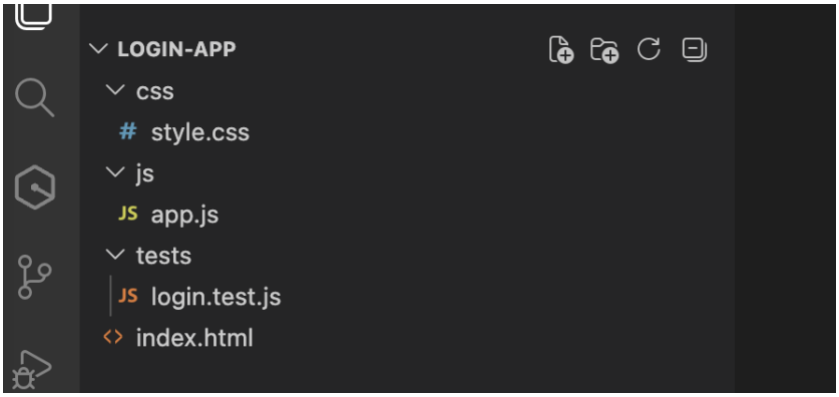
# FASE 1 – Página Web de Login (HTML/CSS/JS)

| ⏱ Duración | 2 sesiones (4 horas) |
|---|---|

En esta fase crearemos una página de login funcional y con validación en el cliente. No requiere backend; toda la lógica reside en JavaScript.

### 1.1 Estructura del proyecto

Crea la siguiente estructura de carpetas en tu equipo:




### 1.2 index.html

Página principal con formulario de login accesible (atributos ARIA incluidos).

### 1.3 css/style.css

Diseño limpio con variables CSS y modo oscuro preferido del sistema.

### 1.4 js/app.js

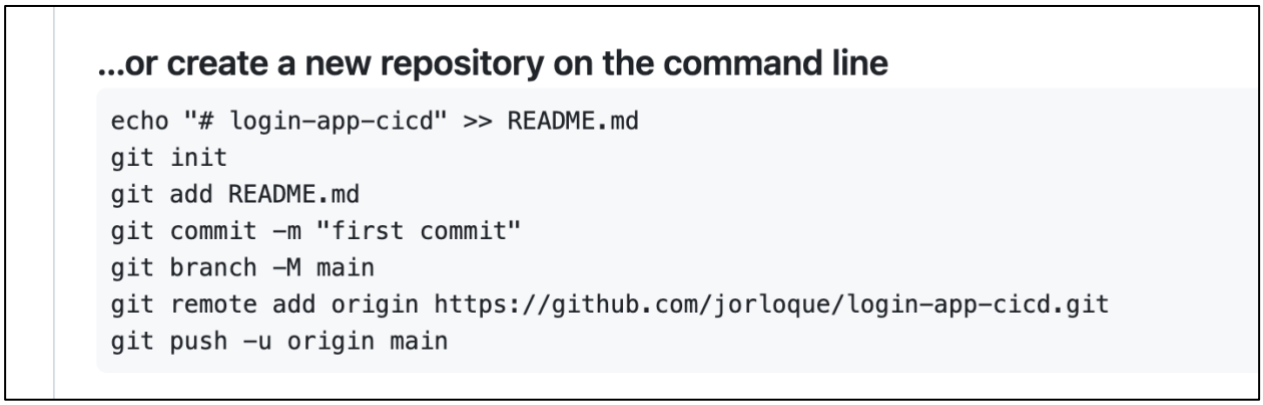
Validación en el cliente y simulación de autenticación.

Los archivos se encuentran en una carpeta para descargarlos.

# FASE 2 – Repositorio GitHub y primer commit

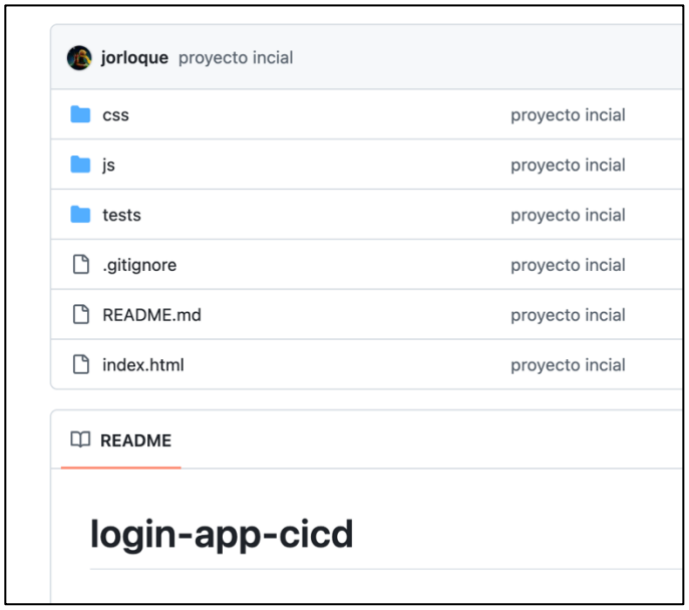
## 2.1 Crear el repositorio en GitHub

1. Accede a https://github.com → New repository.
2. Nombre: login-app-cicd
3. Clic en Create repository.




### 2.2 Subir el código

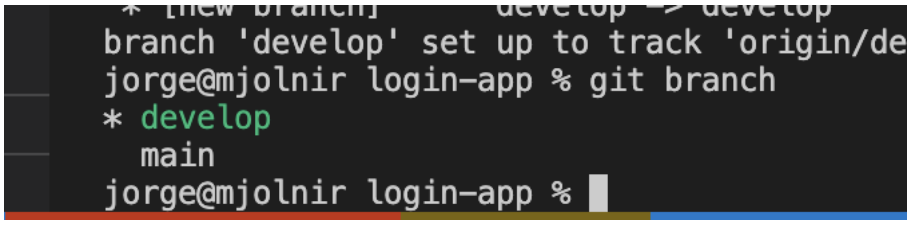
```
git add .
git commit -m "proyecto inicial"
git push origin main
```



## 2.3 Estrategia de ramas (GitFlow simplificado)

| Rama | Propósito | Regla |
|---|---|---|
| main | Código en producción | Solo merge vía Pull Request aprobado |
| develop | Integración de features | Merge tras pasar CI |
| feature/* | Nueva funcionalidad | Borrar tras merge |
| fix/* | Corrección de bugs | Hotfix urgente |

```
# Crear y publicar la rama develop
git checkout -b develop
git push -u origin develop
```



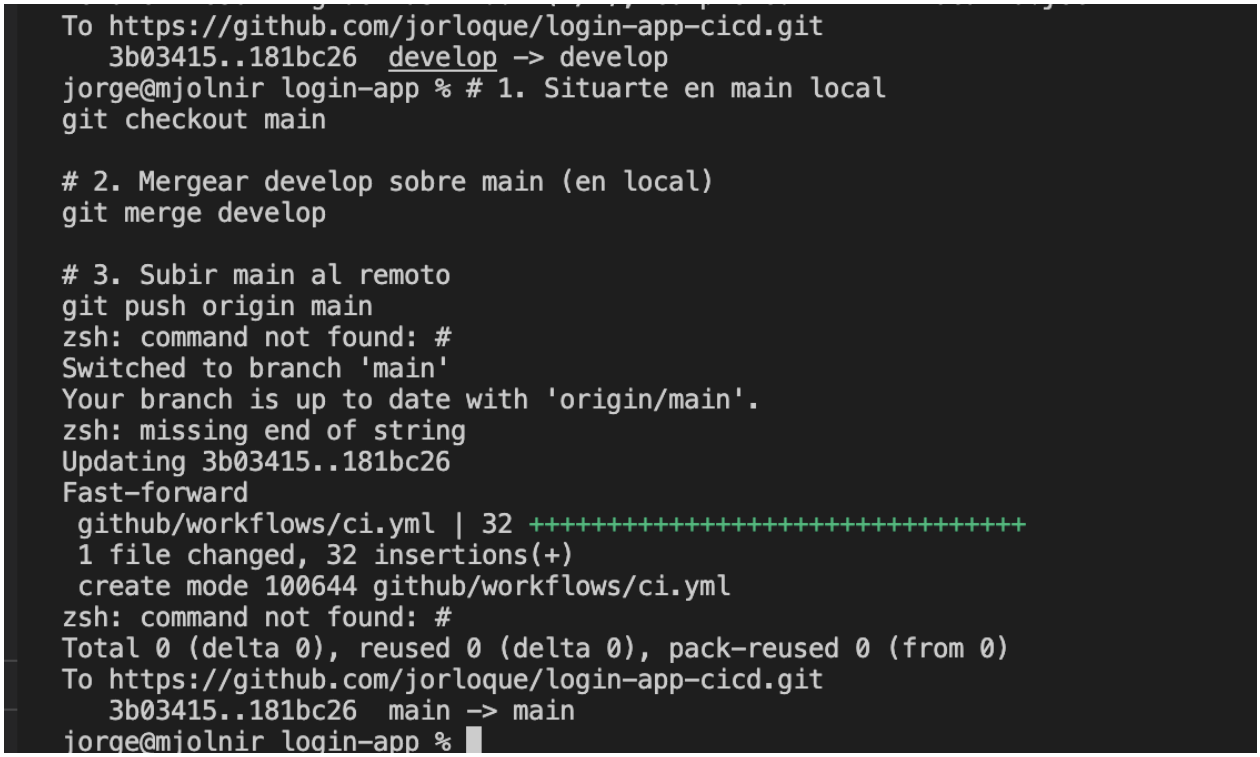
# FASE 3 – Validaciones y Testing con GitHub Actions

## 3.1 Conceptos clave de GitHub Actions

| Término | Descripción |
|---|---|
| Workflow | Archivo YAML en .github/workflows/ que define la automatización completa |
| Job | Conjunto de pasos que se ejecutan en el mismo runner (máquina virtual) |
| Step | Comando individual o Action de terceros dentro de un Job |
| Runner | Servidor donde corre el job (ubuntu-latest, windows-latest, macos-latest) |
| Trigger (on) | Evento que dispara el workflow: push, pull_request, schedule, workflow_dispatch… |
| Secret | Variable cifrada del repositorio: Settings → Secrets and variables → Actions |

## 3.2 Workflow de CI – Validación de HTML y linting

Crea el archivo .github/workflows/ci.yml.

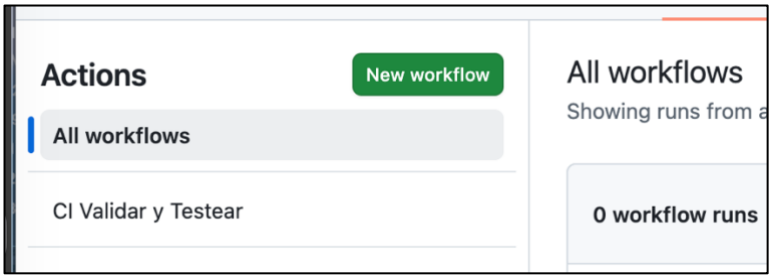



Como las ramas se habrán quedado descompasadas podemos o bien hacer el compare y PR desde github, aunque aquí luego habría que hacer un pull del repositorio al repositorio local.

```
git checkout main
git pull origin main
```

O realizar la opción de unir las ramas y luego hacer un push que es la opción por la cual se ha optado.

A continuación, probaremos el Action para ver qué se ejecuta.

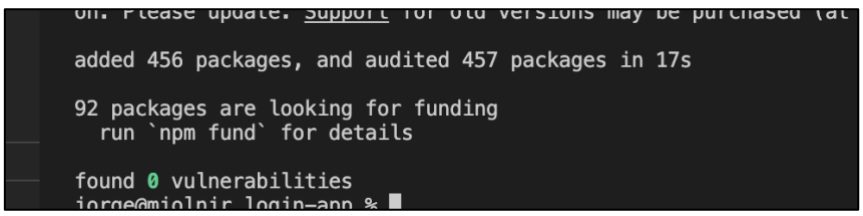



Nos saltará el siguiente error que arreglaremos más adelante.




## 3.3 Inicializar proyecto Node.js para los tests

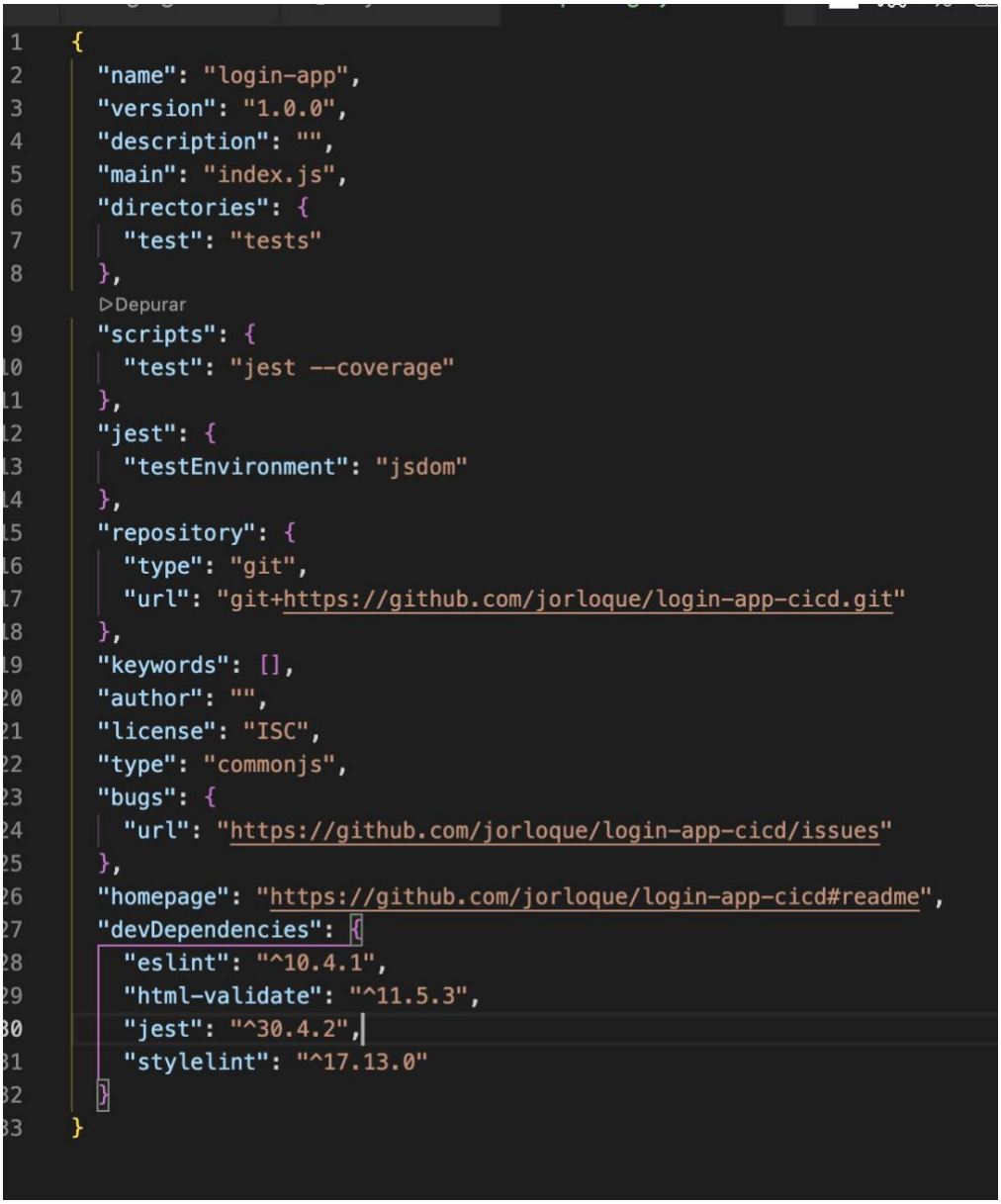
```
npm init -y
npm install --save-dev jest eslint stylelint html-validate
```



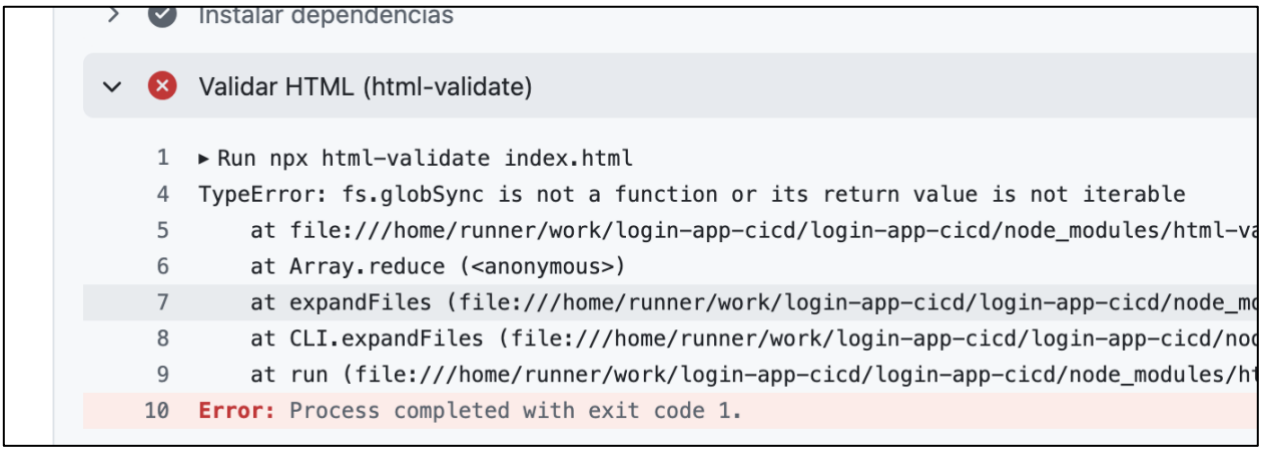
Añadir en package.json:

```
{
  "scripts": {
    "test": "jest --coverage"
  },
  "jest": {
    "testEnvironment": "jsdom"
  }
}
```

Quedando algo parecido a la siguiente imagen:



Ahora nos sale el mensaje de error de validación del html, pero esto es porque aún no lo hemos configurado.




## 3.4 Tests unitarios con Jest (tests/login.test.js)

> ⚠ **Nota** Comprueba que para que los tests puedan importar validate(), exporta la función en app.js: module.exports = { validate }; y que esta línea aparezca en el archivo.

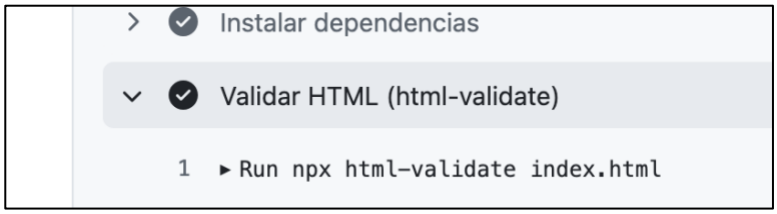
Puede ser que tenga problemas al ejecutar el runner, para ello en local:

```
npm install --save-dev html-validate@9
git add package.json package-lock.json
git commit -m "fix: downgrade html-validate to v9 for Actions compatibility"
git push origin main
```

Y ya si sigue dando error hay que regenerar el proyecto:

```
# Elimina el lock file y node_modules para empezar limpio
rm package-lock.json
rm -rf node_modules
# Reinstala todo y genera un lock file limpio
npm install
# Sube el lock file actualizado
git add package-lock.json
git commit -m "fix: regenerate package-lock.json"
git push origin main
```

Con esto el apartado de lint html ya estaría completo:




En el linting CSS sigue dando error, para ello stylelint necesita su archivo de configuración. Crea .stylelintrc.json en la raíz del proyecto:

```
npm install --save-dev stylelint-config-standard
```

Y reharemos todo otra vez

```
rm package-lock.json
rm -rf node_modules
npm install
git add package-lock.json
git commit -m "fix: regenerate package-lock.json with stylelint-config-standard"
git push origin main
```

Puede ser que siga dando error, entonces crearemos el fichero a mano con su contenido y lo subiremos al repositorio.

```
Crear .stylelintrc.json
```

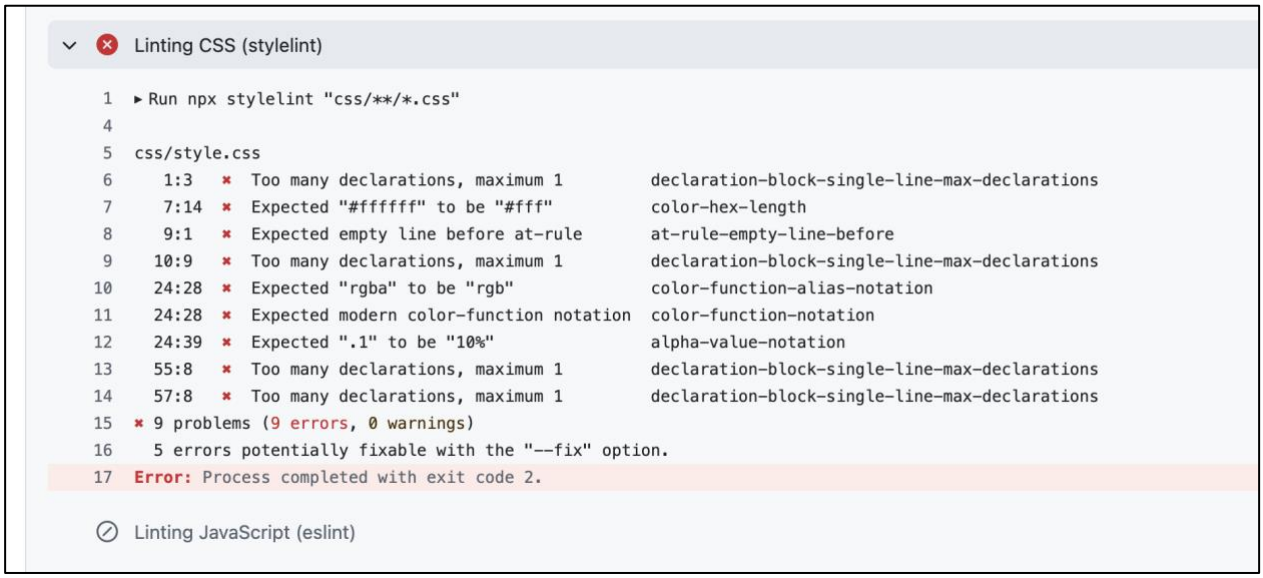
Añadir

```
{
  "extends": ["stylelint-config-standard"],
  "rules": {
    "no-descending-specificity": null
  }
}
```

Y guardar:

```
git add .stylelintrc.json
git commit -m "fix: add .stylelintrc.json config file"
git push origin main
```

Con esto ya nos aparece otro tipo de error propio del linter del css:




Primero arreglar automáticamente los más rápidos:

```
npx stylelint "css/**/*.css" --fix
```

Esto corrige algunos errores solo. Luego súbelo:

```
git add css/style.css
git commit -m "fix: auto-fix stylelint errors"
git push origin main
```

A continuación:

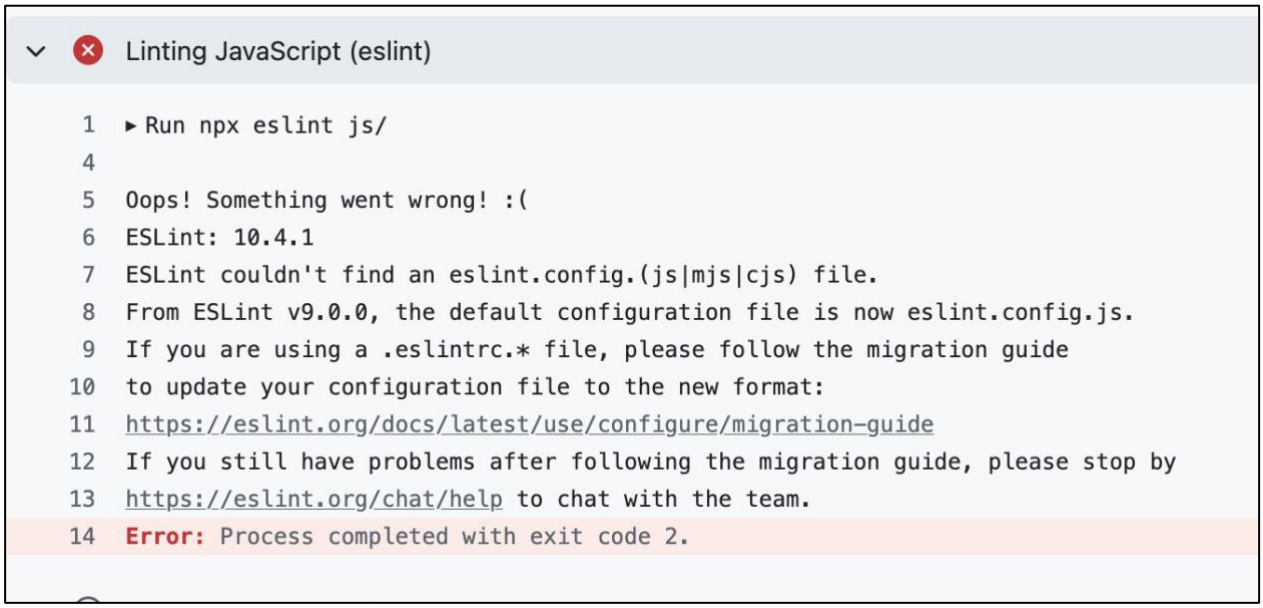
Las líneas afectadas son la 1, 11, 56 y 58. Por ejemplo si tienes algo así:

css

```
/* Antes */
* { box-sizing: border-box; margin: 0; padding: 0; }
```

```
/* Después */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Indentando bien las líneas ahora pasaremos al linting de Javascript:




Mismo problema que antes, falta el archivo de configuración. ESLint v9+ usa un formato nuevo. Crea el archivo eslint.config.js en la raíz del proyecto desde VS Code:

```
js
import js from "@eslint/js";

export default [
  js.configs.recommended,
  {
    rules: {
      "no-unused-vars": "warn",
      "no-undef": "warn"
    }
  }
];
```

Instala la dependencia necesaria:

```
bash
npm install --save-dev @eslint/js
```

Pero como tu package.json tiene "type": "commonjs" el import no funcionará. Usa en su lugar el formato CommonJS renombrando el archivo a eslint.config.cjs:

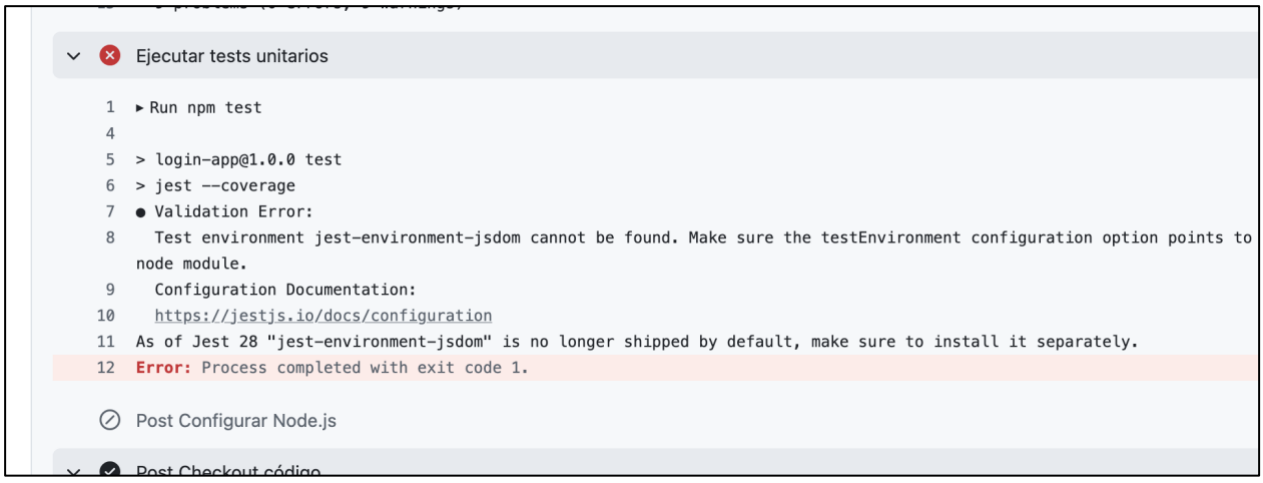
```
js
const js = require("@eslint/js");

module.exports = [
  js.configs.recommended,
  {
    rules: {
      "no-unused-vars": "warn",
      "no-undef": "warn"
    }
  }
];
```

Luego sube todo:

```
bash
rm package-lock.json
rm -rf node_modules
npm install
git add eslint.config.cjs package.json package-lock.json
git commit -m "fix: add eslint config for v9"
git push origin main
```

Con esto ya podemos pasar a validar los tests unitarios:




Falta instalar jest-environment-jsdom por separado. Desde Jest 28 ya no viene incluido:

```
bash
npm install --save-dev jest-environment-jsdom
```

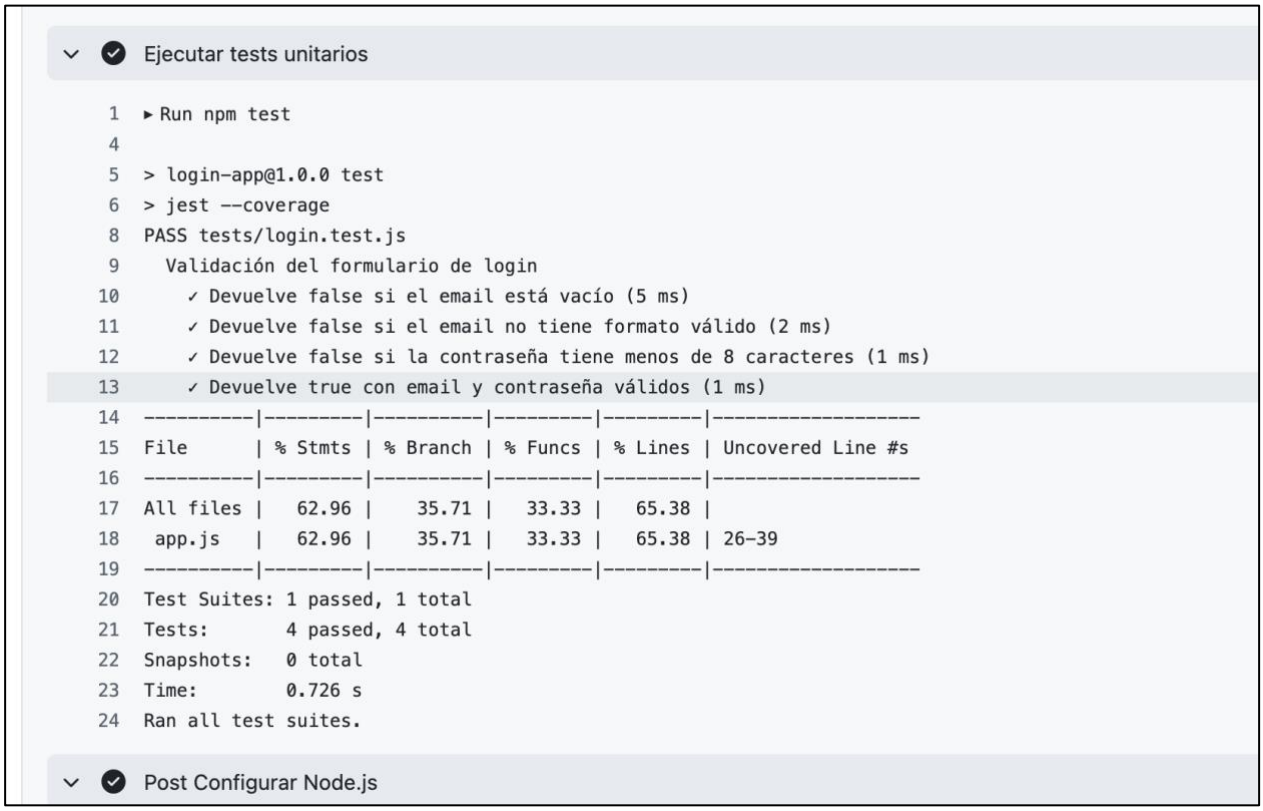
Luego regenera el lock file y sube:

```
bash
git add package.json package-lock.json
git commit -m "fix: add jest-environment-jsdom"
git push origin main
```

Puede ser que nos salga el mismo error de siempre:

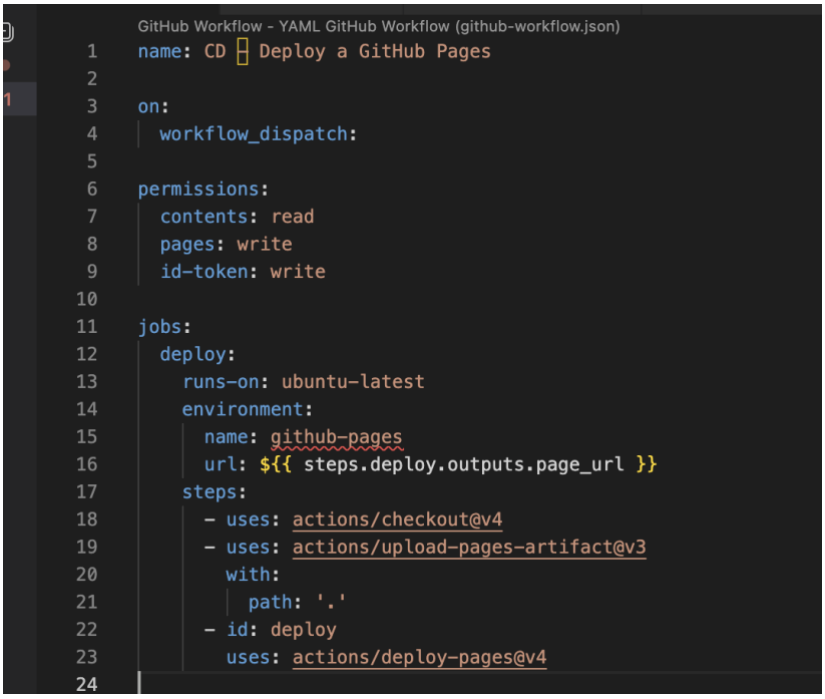
```
rm package-lock.json
rm -rf node_modules
npm install
git add package.json package-lock.json
git commit -m "fix: regenerate package-lock.json"
git push origin main
```

Si todo hay correcto deberíamos ver algo parecido a la siguiente imagen:



## 3.5 Workflow de CD – GitHub Pages (despliegue automático)

Activa GitHub Pages en: Settings → Pages → Source: GitHub Actions.




```yaml
name: CD - Deploy a GitHub Pages

on:
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deploy.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-pages-artifact@v3
        with:
          path: '.'
      - id: deploy
        uses: actions/deploy-pages@v4
```

Para mejorar esto deberíamos lanzar el workflow de CD cuando finalice correctamente el de CI.

```yaml
name: CD - Deploy a GitHub Pages

on:
  workflow_run:
    workflows: ["CI Validar y Testear"]
    types:
      - completed
    branches:
      - main

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    environment:
      name: github-pages
      url: ${{ steps.deploy.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/upload-pages-artifact@v3
        with:
          path: '.'
      - id: deploy
        uses: actions/deploy-pages@v4
```

# TAREA FINAL – Copia automática de archivos a Amazon S3

Como tarea final, se ampliará el pipeline con un nuevo workflow de GitHub Actions que, en cada **push** a la rama main, copie automáticamente los archivos de la web a un **bucket de Amazon S3**. De esta forma, el bucket siempre contendrá la última versión del código subida al repositorio.

**Objetivos de la tarea:**

- Crear y configurar un bucket de Amazon S3.
- Gestionar credenciales de forma segura mediante los Secrets de GitHub.
- Utilizar una Action oficial de AWS (aws-actions/configure-aws-credentials) dentro de un workflow.
- Sincronizar los archivos del repositorio con el bucket en cada push.

## T.1 Crear el bucket en S3

Accede a la consola de AWS (cuenta propia o AWS Academy Learner Lab) → S3 → Crear bucket. El nombre debe ser único a nivel mundial, por ejemplo: login-cicd-tunombre-2627. Selecciona la región us-east-1 y deja el resto de opciones por defecto (acceso público bloqueado).

También se puede crear desde la terminal con AWS CLI:

```
aws s3 mb s3://login-cicd-tunombre-2627 --region us-east-1
```

## T.2 Obtener las credenciales de AWS

El workflow necesita credenciales para escribir en el bucket. Según el tipo de cuenta:

- **Cuenta propia de AWS:** crea un usuario en IAM con una política que solo permita trabajar sobre el bucket (principio de mínimo privilegio) y genera una clave de acceso (Access key ID y Secret access key).
- **AWS Academy Learner Lab:** inicia el laboratorio y en AWS Details → AWS CLI → Show copia aws_access_key_id, aws_secret_access_key y aws_session_token. Estas credenciales caducan al cerrar la sesión del laboratorio, por lo que habrá que actualizarlas en GitHub cada vez que se inicie de nuevo.

Política IAM de ejemplo (solo para cuenta propia; sustituye el nombre del bucket):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::login-cicd-tunombre-2627"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::login-cicd-tunombre-2627/*"
    }
  ]
}
```

## T.3 Guardar las credenciales como Secrets en GitHub

Las credenciales nunca deben escribirse en el código ni en el workflow. En el repositorio, ve a Settings → Secrets and variables → Actions → New repository secret y crea los siguientes secretos:

- **AWS_ACCESS_KEY_ID** – identificador de la clave de acceso.
- **AWS_SECRET_ACCESS_KEY** – clave secreta.
- **AWS_SESSION_TOKEN** – solo si se usa AWS Academy Learner Lab.
- **AWS_REGION** – región del bucket, por ejemplo us-east-1.
- **S3_BUCKET** – nombre del bucket, por ejemplo login-cicd-tunombre-2627.

## T.4 Crear el workflow .github/workflows/s3.yml

Crea el archivo .github/workflows/s3.yml con el siguiente contenido:

```
name: Copia a S3

on:
  push:
    branches: [ main ]

jobs:
  deploy-s3:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar el código del repositorio
        uses: actions/checkout@v4

      - name: Configurar credenciales de AWS
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-session-token: ${{ secrets.AWS_SESSION_TOKEN }}
          aws-region: ${{ secrets.AWS_REGION }}

      - name: Copiar los archivos al bucket
        run: |
          aws s3 sync . s3://${{ secrets.S3_BUCKET }} --delete \
            --exclude ".git/*" \
            --exclude ".github/*" \
            --exclude "node_modules/*" \
            --exclude "tests/*" \
            --exclude "package*.json" \
            --exclude "*.cjs"

      - name: Mostrar el contenido del bucket
        run: aws s3 ls s3://${{ secrets.S3_BUCKET }} --recursive
```

**Explicación de los pasos:**

- **actions/checkout@v4** descarga en la máquina virtual los archivos del commit que ha provocado el push.
- **aws-actions/configure-aws-credentials@v4** es la Action oficial de AWS: lee los Secrets y deja configurada la AWS CLI para los pasos siguientes. Si no usas AWS Academy, elimina la línea aws-session-token.
- **aws s3 sync** compara el repositorio con el bucket y solo sube los archivos nuevos o modificados en el push. La opción --delete borra del bucket los archivos que se hayan eliminado del repositorio, y --exclude evita subir archivos que no forman parte de la web.
- El último paso lista el contenido del bucket para comprobar el resultado en el log de la Action.

## T.5 Subir el workflow y comprobar el resultado

```
git add .github/workflows/s3.yml
git commit -m "feat: copia automática de archivos a S3"
git push origin main
```

En la pestaña Actions del repositorio debe aparecer el workflow «Copia a S3» en verde. Comprueba en la consola de S3 que el bucket contiene index.html, la carpeta css y la carpeta js. Después, modifica algún archivo (por ejemplo, el texto del botón de login), haz un nuevo push y verifica que el archivo se ha actualizado en el bucket.

## T.6 Ampliación (opcional)

- Activa Static website hosting en el bucket y configura una política de lectura pública para acceder a la página de login desde la URL del sitio web de S3.

## T.7 Entregables

- Captura del bucket creado en la consola de S3.
- Captura de los Secrets configurados en GitHub (solo los nombres, nunca los valores).
- Archivo s3.yml y captura de la ejecución correcta en la pestaña Actions.
- Captura del contenido del bucket antes y después de un segundo push con cambios.